PROBLEMAS DE SISTEMAS ELECTRÓNICOS BASADOS EN MICROPROCESADOR
TEMA 4 – Estudio del procesador ARM7

1. Diseña una subrutina, de nombre **bitrev**, que invierta el orden de los bits de una palabra contenida en un registro.

2. Suponer que en un registro se encuentra almacenado un carácter en código ASCII. Diseña una subrutina, llamada **toupper**, tal que si se trata de una letra minúscula la pase a mayúscula. En el caso de que el carácter ASCII en el registro no sea una letra minúscula, la subrutina no hará nada.

   Tabla con el código ASCII:

ASCII	Hex	Símbolo	ASCII	Hex	Símbolo	ASCII	Hex	Símbolo	ASCII	Hex	Símbolo	ASCII	Hex	Símbolo	ASCII	Hex	Símbolo	ASCII	Hex	Símbolo
0	0	NUL	16	10	DLE	32	20	(space)	48	30	0	64	40	@	80	50	P	96	60	`	112	70	p
1	1	SOH	17	11	DC1	33	21	!	49	31	1	65	41	A	81	51	Q	97	61	a	113	71	q
2	2	STX	18	12	DC2	34	22	"	50	32	2	66	42	B	82	52	R	98	62	b	114	72	r
3	3	ETX	19	13	DC3	35	23	#	51	33	3	67	43	C	83	53	S	99	63	c	115	73	s
4	4	EOT	20	14	DC4	36	24	$	52	34	4	68	44	D	84	54	T	100	64	d	116	74	t
5	5	ENQ	21	15	NAK	37	25	%	53	35	5	69	45	E	85	55	U	101	65	e	117	75	u
6	6	ACK	22	16	SYN	38	26	&	54	36	6	70	46	F	86	56	V	102	66	f	118	76	v
7	7	BEL	23	17	ETB	39	27	'	55	37	7	71	47	G	87	57	W	103	67	g	119	77	w
8	8	BS	24	18	CAN	40	28	(	56	38	8	72	48	H	88	58	X	104	68	h	120	78	x
9	9	TAB	25	19	EM	41	29	)	57	39	9	73	49	I	89	59	Y	105	69	i	121	79	y
10	A	LF	26	1A	SUB	42	2A	*	58	3A	:	74	4A	J	90	5A	Z	106	6A	j	122	7A	z
11	B	VT	27	1B	ESC	43	2B	+	59	3B	;	75	4B	K	91	5B	[	107	6B	k	123	7B	{
12	C	FF	28	1C	FS	44	2C	,	60	3C	<	76	4C	L	92	5C	\	108	6C	l	124	7C	|
13	D	CR	29	1D	GS	45	2D	-	61	3D	=	77	4D	M	93	5D	]	109	6D	m	125	7D	}
14	E	SO	30	1E	RS	46	2E	.	62	3E	>	78	4E	N	94	5E	^	110	6E	n	126	7E	~
15	F	SI	31	1F	US	47	2F	/	63	3F	?	79	4F	O	95	5F	_	111	6F	o	127	7F	

3. Diseña una subrutina, de nombre **divi**, para hacer la división de dos números sin signo. Se le pasará el dividendo en r0 y el divisor en r1, y devolverá el cociente en r0 y el resto en r1.

4. Escribe una subrutina llamada **bin_hex_conv** que convierta un número de 16 bits (4 cifras hexadecimales), alojado en la parte baja de un registro, en su equivalente en hexadecimal, escrito con caracteres ASCII (4 caracteres ASCII - 32 bits), también en un registro.

5. Se desea diseñar una subrutina, llamada **bin_asc_conv**, que convierta un dato binario de 32 bits contenido en un registro en su equivalente en ASCII, es decir, en un conjunto de 32 bytes '0' o '1' en ASCII y los escriba en memoria a partir de una determinada dirección hacia posiciones crecientes. Concretamente, en la posición indicada aparecerá el bit de peso 0, en la siguiente el de peso 1, y así sucesivamente hasta escribir los 32 bits, cada uno de ellos en ASCII.

   Se pide:
   - Indicar cuántos parámetros se pasan y en qué registros.
   - Indicar cuántos resultados se devuelven al programa principal y en qué registros.
   - Escribir la subrutina. Comentarla adecuadamente.

6. Escribir una subrutina en ensamblador de ARM7, llamada **rms2**, para el cálculo del cuadrado del valor "rms" de una señal muestreada almacenada en la memoria:

$$x_{rms}^2 = \frac{1}{n}\sum_{i=0}^{n-1} x_i^2$$

   Suponer que las muestras x_i son datos de 16 bits (números sin signo) que se encuentran en posiciones consecutivas de memoria a partir de una dirección almacenada en r0, y que su número total n es una potencia exacta de 2 almacenada en r1 ($n=2^{r1}$). La subrutina devolverá el valor x^2_{rms} en el registro r0. Para simplificar, suponer que en el cálculo no se producen situaciones de desbordamiento.

7. Se desea diseñar una subrutina llamada **reverse** que almacene los elementos de una cadena de caracteres ASCII en memoria en las mismas posiciones, pero en orden inverso. Tener en cuenta que una cadena de caracteres debe acabar siempre con el carácter nulo (0x00) y, por tanto, se debe invertir el orden de todos los caracteres salvo el nulo, que permanecerá en su posición. Como parámetro de entrada se tiene el puntero al primer carácter de la cadena.

8. Diseña una subrutina llamada **memcopy** que tome una matriz de N elementos de 32 bits, almacenados en posiciones consecutivas de la memoria a partir de una dirección M1, y la copie a partir de otra dirección M2, también en posiciones consecutivas. Suponer que N se pasa en el registro r2, M1 en r1 y M2 en r0.

9. Diseña una versión mejorada (de ejecución más rápida) de la subrutina **memcopy**, de nombre **memcopyf**, mediante el uso de las instrucciones de transferencia múltiple LDM y STM con 8 registros. Estimar la mejora del rendimiento de **memcopyf** respecto a **memcopy**.

10. Escribir una subrutina en ensamblador de ARM7 llamada **MCD** que calcule el Máximo Común Divisor (MCD) de dos números naturales de 32 bits utilizando el algoritmo tradicional de Euclides. Dicho algoritmo se explica a continuación:

    Para calcular el MCD de dos números se divide el mayor (dividendo, D0) entre el menor (divisor, d0) y se comprueba si el resto (r0) es igual o no a 0. En caso de que sea igual a 0, el MCD es el divisor. En caso contrario, se vuelve a dividir ahora el divisor (D1=d0) entre el resto (d1=r0) y se vuelve a comprobar si el nuevo resto (r1) es 0 o no. Así se hace sucesivamente hasta que el resto sea igual a 0. En ese caso, el MCD será el último divisor.

    <u>Ejemplo</u>: calcular el máximo común divisor de 2366 y 273. Según el algoritmo de Euclides:
    a) Se determina cuál es el número mayor, en este caso 2366.
    b) Iteración 0: comenzamos con D0=2366 y d0=273. Al dividir D0/d0, se obtiene 8 de cociente y el resto es r0=182.
    c) Iteración 1: como el resto r0 no ha sido 0, se toma como nuevo dividendo D1 el anterior divisor d0 (D1=273) y como nuevo divisor d1 el anterior resto r0 (d1=182). Al dividir D1/d1, se obtiene 1 de cociente y el resto es r1=91.
    d) Iteración 2: como el resto r1 no ha sido 0, se toma como nuevo dividendo D2 el anterior divisor d1 (D2=182) y como nuevo divisor d2 el anterior resto r1 (d2=91). Al dividir D2/d2, se obtiene 2 de cociente y el resto es r2=0.
    e) Como el resto r2 ha sido 0, el MCD es el último divisor (d2), es decir, 91.

    Utilizar la subrutina **divi** para hacer las divisiones. Usar correctamente el convenio de paso de parámetros a subrutinas.